



Assessed Coursework

Course Name	Web Application Systems			
Coursework Number	3 (of 4) – Group Project Web App			
Deadline	Time:	10.00pm	Date:	20 March 2026
% Contribution to final course mark	25	This should take at most this many hours:		20
Solo or Group ✓	Solo		Group	✓
Submission Instructions	Via Moodle – see Page 4			
Who Will Mark This? ✓	Lecturer	Tutor ✓	Other	
Feedback Type? ✓	Written ✓	Oral	Both	
Individual or Generic? ✓	Generic	Individual ✓	Both	
Other Feedback Notes				
Please Note: This Coursework cannot be Re-Done				

Code of Assessment Rules for Coursework Submission

Deadlines for the submission of coursework which is to be formally assessed will be published in course documentation, and work which is submitted later than the deadline will be subject to penalty as set out below. The primary grade and secondary band awarded for coursework which is submitted after the published deadline will be calculated as follows:

- (i) in respect of work submitted not more than five working days after the deadline
 - a. the work will be assessed in the usual way;
 - b. the primary grade and secondary band so determined will then be reduced by two secondary bands for each working day (or part of a working day) the work was submitted late.
- (ii) work submitted more than five working days after the deadline will be awarded Grade H.

Penalties for late submission of coursework will not be imposed if good cause is established for the late submission. You should submit documents supporting good cause via MyCampus.

Penalty for non-adherence to Submission Instructions is 2 bands

Marking Criteria

See Page 5

Web Application Systems

Group Project Web App (15%)

Introduction

Your WAS Project is mainly based on the development of a web application of your choosing. Your web application should be developed using Python, Django, HTML, CSS and associated technologies including JavaScript, JQuery and AJAX.

Implementation of your web app should draw on the skills built up during the development of Rango. Teams are free to use their own ideas, however there are some basic expectations:

- the app should involve user authentication;
- it should certainly interact with some kind of model stored in a database;
- it should be visually appealing and have an intuitive user interface;
- overall the functionality supported should be rich enough in order to allow you to demonstrate an understanding of the technologies listed above.

Beyond these guidelines, it is up to you, though if you do need some ideas, the following section lists some example projects from students in similar courses in previous years.

Example project ideas from previous years

- *Rate My Beard*
 - Users with beards can upload photos inviting feedback; other users can login, view photos and give feedback
- *GU Grub Guide*
 - Users can leave ratings for restaurants and view other users' ratings
- *UnNoobMe*
 - Students seeking private tutors and tutors seeking clients are allocated to each other
- *Does my MSP Represent Me?*
 - Users can find out information about how their MSP is voting in Parliament
- *TripShare*
 - Users can post information about their upcoming trips, browse / search for trips and request to join a specific trip
- *Bargain Radar*
 - Small businesses can post their discounted deals; customers can browse deals and add them to baskets
- *SHAKESbeer*
 - Allows users to contribute cocktail recipes; users can browse and search over, and rate and comment on, cocktails
- *FilmBook*
 - Users can login and browse, search and rate movies; film producers can login and upload new films so that they can be rated
- *Federated Health Search Application*
 - People find out about particular conditions and to save the information that they find into different folders. The app lets people search across two different medical sites
- *Zombie Survival*
 - A game of search and survival. The web application needs to interface with a series of classes that create the world in which the player needs to survive.

Working in a team

You should think about how the tasks associated with the development of your team's web app should be divided. Try to ensure that activities are assigned so that every member of the team can be involved at all times. Here is a possible breakdown of responsibilities for a four-person team (this is just an example and you should feel free to organise things differently if you prefer. Remember, contributions can also be balanced with earlier work in the Design Specification submission):

1. user authentication, unit testing
2. models, JavaScript, AJAX
3. views and URLs
4. templates and CSS

What should be submitted

You should ensure that the latest version of the code is stored in a public GitHub repo, and that the hosted project works (try cloning to a new machine). The URL for your GitHub will need to be submitted. You should ensure that your final application addresses the following requirements:

Deployment

- Your GitHub repo will be checked via the submitted URL – make sure the repo is public!
- Requirements file is included and contains the correct packages
 - Within your main project folder you should include a file called `requirements.txt` that can be used to install the relevant packages within a newly-created virtual environment using `pip install -r requirements.txt`. You can create this file easily using `pip freeze > requirements.txt`.
- Database files not included
 - You should not commit your database files so that when your tutor works on deploying your application on his/her local machine, the database can be recreated from scratch by making the relevant migrations and running your population script.
- Population script works and contains useful example data
 - Create a population script called `population_script.py` with enough relevant data in your main project folder.
- Application can be deployed on marker's own machine

Functionality

- Main functionality has been implemented (hopefully reflecting the design)
 - Here you should refer back to your design specification to guide development towards providing functionality consistent with what was envisaged.
 - If you have used external sources, you should acknowledge them in a README file in the top-level folder of your project. Failure to acknowledge external sources could result in a deduction of marks under the “main functionality” heading.
 - Test that the project as it exists on the repository will definitely work when it is cloned – the main functionality will be checked with respect to the version that your tutor deploys locally (i.e., the version on localhost).
- Application is bug-free / no error messages occur
 - The user should be able to access all of the functionality of the application without encountering Django error messages. You should thoroughly test your application to ensure that this is the case.
- Application includes some JavaScript / JQuery / AJAX
 - Your application should not be based solely on Python and Django, and there should be evidence of usage of one or more of some of the technologies covered later in the course including JavaScript / JQuery / AJAX. (More marks for more ambition!)

Look and Feel

- Polished / refined interface, not clunky
- Uses a responsive CSS framework
 - CSS can be quite fiddly – to help with the layout and appearance of your web pages you are recommended to use a CSS framework such as Bootstrap.
- If browser window size changed, is the change handled neatly?
- **In the “Look and Feel” category, marks are available for an attractive and intuitive user interface with good use of CSS.**

Code

- Templates inherit from base
- URLs are relative, i.e., use the {% url ..%} tag
- Code contains helper functions/classes (if required)
- Code is readable, clear and commented where appropriate
- CSS and JavaScript are kept separate from templates (i.e., not inline)
- No repetition of code blocks in the views or templates
- Unit tests are included
- **In the “Code” category, marks are available for well-organised code, adhering to the principles that you will have learned during the development of the Rango app.**

A sample marking scheme is included in Appendix 1.

Use of external sources

It is recognised that you may use external sources (e.g., Bootstrap) when developing your application. However these *must* be acknowledged when you submit (see below).

How to submit

After developing your web application you should ensure that the latest version of your code has been pushed to your team’s GitHub repo.

One member of the team should then submit a one-page document via the “Project” submission icon on the Moodle page for the course, containing the following information:

- Name of web application
- Lab group number and team letter (e.g., Lab group 4, Team B)
- Team members (names and student numbers)
- GitHub URL for the app
- Any external sources used (code, libraries, APIs), if applicable.

The person making the submission will be required to complete a Declaration of Originality on behalf of all team members when submitting via Moodle. If you have used any external sources, be sure to acknowledge them in your one-page document. For reference, the School’s plagiarism policy is contained in Appendix A of the Undergraduate Class Guide.

Appendix 1: Sample marking scheme

The following marking scheme is intended to give a broad indication as to how marks will be apportioned. The actual marking scheme used in practice may deviate slightly from this.

Category	Marks
Deployment	
Requirements file is included and contains the correct packages	2
Database files not included	1
Population script works and contains useful example data	6
Application can be deployed on marker's own machine	3
Functionality	
Site core functionality – an ambitious project has been undertaken	20
Application is bug-free / no error messages occur	2
Application includes some JavaScript / JQuery / AJAX	6
Look and Feel	
Polished / refined interface, not clunky	8
Uses a responsive CSS framework	3
If browser window size changed, is the change handled neatly?	1
Code	
Templates make use of inheritance	2
URLs are relative, i.e., use the {% url ..%} tag	2
Code structure: readable, clear and commented where appropriate etc	3
CSS and JavaScript are kept separate from templates (i.e., not inline)	3
No repetition of code blocks in the views or templates	2
Unit tests are included and are comprehensive, including testing views	6
Total	70

The total mark will be converted to a band which will be the team's mark for this component of the assessment.